

Views

Três leituras salvas em 08_etapa2_views.sql e expostas por caminhos canônicos

Leitura estável sobre o esquema atual

As views são recriadas com DROP VIEW IF EXISTS seguido de CREATE VIEW. CREATE OR REPLACE exige preservar nomes, tipos e ordem das colunas, condição pouco prática enquanto a consulta ainda evolui. O DROP não usa CASCADE; uma dependência futura deve interromper a instalação em vez de ser removida sem aviso.

vw_pacientes_internados

A view devolve quem está internado agora, definido pela internação mais recente de cada paciente. **DISTINCT ON (id_paciente)**, junto do ORDER BY por entrada e id decrescentes, escolhe uma linha por pessoa e resolve empates no mesmo instante. Só depois dessa escolha o filtro exige data_hora_saida nula.

```
SELECT DISTINCT ON (i.id_paciente) ...  
  FROM Internacao i  
 ORDER BY i.id_paciente,  
          i.data_hora_entrada DESC,  
          i.id_internacao DESC;
```

O critério evita que uma internação antiga, deixada sem alta, esconda a situação mais recente. A coluna tempo_internado usa CURRENT_TIMESTAMP e muda a cada leitura. Com o seed, aparecem Carla Mendes, Ana Souza e Elisa Rocha. A rota é **GET /pacientes/internados**.

vw_residentes_sem_supervisor

A consulta lista escalas cujo preceptor não tem titulação de doutor. O esquema atual exige id_preceptor, mas os LEFT JOIN preservam o tratamento do caso sem vínculo se a nulabilidade mudar. A coluna motivo diferencia ausência de preceptor e titulação insuficiente.

A comparação é feita com o domínio canônico: **pre.titulacao IS DISTINCT FROM 'doutor'**. O schema da API normaliza a entrada e o trigger protege escritas diretas no banco. Por isso descrições de doutorado não geram falso positivo. O seed devolve quatro escalas. A rota é **GET /residentes/sem-supervisor-doutor**.

vw_estatisticas_atendimentos_mensal

A terceira view agrupa por mês e unidade. Ela informa total de atendimentos, duração média, menor e maior duração e até três procedimentos mais frequentes. O ranking exige três CTEs: **base** define o grupo, **totais** calcula as estatísticas e **contagem** numera os procedimentos com ROW_NUMBER.

Estrutura e resultado

CTE	Responsabilidade
base	Seleciona atendimento, mês, unidade e duração; registros sem unidade ficam fora.
totais	Conta atendimentos e calcula média, mínimo e máximo sem multiplicar linhas.
contagem	Agrupa procedimentos e cria um ranking por mês e unidade.

O total nasce em base, antes da junção com Procedimento_Realizado. Essa ordem evita contar o mesmo atendimento várias vezes. `string_agg` reúne os três primeiros procedimentos e usa a posição do ranking; empates são ordenados pelo nome.

Mês e unidade no seed	Atendimentos	Duração média
2026-07 Ambulatorio	2	37,5 min
2026-07 Enfermaria A	3	45,0 min
2026-07 Pronto-Socorro	3	26,7 min
2026-07 UTI Adulto	5	52,0 min
2026-06 Enfermaria A	1	30,0 min
2026-06 Pronto-Socorro	1	45,0 min

A rota é **GET /atendimentos/estatisticas-mensais**. Qualquer registro com `id_unidade` nulo fica fora porque não há grupo de unidade a atribuir. A nulabilidade foi mantida para dados anteriores à migração.

Como a API usa as views

As views são somente leitura e não têm chave primária. Em vez de fingir que são entidades mutáveis, os routers executam `SELECT` pela sessão `SQLAlchemy` e validam a resposta com os schemas `Pydantic`. Os caminhos literais são registrados antes das rotas com identificador, evitando que nomes como `internados` ou `estatisticas-mensais` sejam tratados como ids.

tests/test_db.py confere colunas e resultados do seed. **tests/test_http.py** chama os caminhos públicos e verifica que as rotas fixas são resolvidas corretamente. O teste de titulação em **tests/test_adversarial.py** cobre o dado que alimenta a segunda view.